



University of
Nottingham

UK | CHINA | MALAYSIA

Developing Maintainable Software

COMP2013 – Lecture 02

Marjahan Begum and Max L. Wilson



Today's Lecture Slot

**Introduction to
DevOps**

40mins

**Introduction
Continuous
Integration
and
Continuous
Deliver(CI/CD)**

20mins

**Plan for
Lab 2**

10mins

**Extra
What?**

10mins



Learning Outcomes for this Week

- At the end of this week ...
 - Understand what is DevOps in terms of principles
 - Be able to understand and apply role of business and people in the organisation to create values through Dev and Ops
 - Understand how to operationalize DevOps in a software companies through CI/CD
 - Practice some of the DevOps principles through different scenarios



Timetable of Activities



Week	Beginning	Lecture - Tue 2-4pm	Lecture includes...	Lab - Thu 3-6pm
1	Mon 28th Sept	Module Introduction	Industry Experience	Refreshing your OO Skills
2	Mon 5th Oct	Introduction to DevOps	Continuous Integration & Deployment	DevOps Scenario Comparisons
3	Mon 12th Oct	Refactoring 1: (Dev)	Identifying Code Smell in Legacy Code; Swing; CW Release	Refactor 1
4	Mon 19th Oct	Refactoring 2: (Dev)	Techniques to remove code smells;CapitalOne - AI in Industry	Refactor 2
5	Mon 26th Oct	Refactoring 3: (Dev)	Coding principles ,Maintenance Testing ,JavaFx testing Maven intro	Refactor 3
6	Mon 2nd Nov	Refactoring 4 : (Dev)	Design Patterns ; Swing and Java Fx GUIs	Refactor 4
7	Mon 9th Nov	DevOps Infrastructure as Code 1(Ops)	TestFX demo; Java Doc demo ; IntelliJ demo	Refactor 4
8	Mon 16th Nov	DevOps Infrastructure as Code 2(Ops)	Industry Pipeline Demo	CI/CD Pipeline Exercise
9	Mon 23rd Nov	Software Architecture (DevOps)	Industry Panel(tbc)	Code Review Exercise
10	Mon 30th Nov	Advanced DevOps	DevSecOps ; Docker & Containers	Coursework Support
11	Mon 7th Dec	AI in Software Maintenance(DevOps)	Final CW Q&A	Coursework Support

NOTE: some aspects of the timetable are subject to change.

Dev

Ops



I have completed the Lab01

- A. Yes (started, or half completed or finished)
- B. No
- C. Did not know there was a lab sheet

0%
A

0%
B

0%
C



Introduction to DevOps



Job skills (Blackbaud)

Build automation leveraging CI/CD processes, automated testing, unit testing, code coverage and other software development best practices in both public and private cloud technologies



Job skills (Deloitte)

- Create, implement, and manage the development of scripts to automate everyday operations
- Monitor and maintain multiple (DevOps) environments based on client requirements
- Lead program optimization to work within constructs of an AWS environment
- Manage system migrations and system upgrades to create and deploy new cloud environments



Job skills (Rakuten)

- Automate deployments for new services
- Provision infrastructure using Amazon Web Services
- Investigate performance issues
- Assist developers with build and release tasks
- Promote best practices for information security



Agile emphasizes...

- A. Individual and interactions over processes and tools
- B. Follows strict plan
- C. Similarity to waterfall method
- D. Contract negotiation over customer

0

A

0

B

0

C

0

D



Which one is not an agile methodology?

- A. Scrum
- B. Kanban
- C. Lean
- D. Extreme Programming
- E. Minimum Viable Product





Which agile methodology aligns most with DevOps?

- A. Scrum
- B. Kanban
- C. Lean
- D. Extreme Programming
- E. Pair-Programming



The trend toward faster, cheaper, low-risk delivery of software

	1970s–1980s	1990s	2000s–Present
Era	Mainframes	Client/Server	Commoditization and Cloud
Representative technology of era	COBOL, DB2 on MVS, etc.	C++, Oracle, Solaris, etc.	Java, MySQL, Red Hat, Ruby on Rails, PHP, etc.
Cycle time	1–5 years	3–12 months	2–12 weeks
Cost	\$1M–\$100M	\$100k–\$10M	\$10k–\$1M
At risk	The whole company	A product line or division	A product feature
Cost of failure	Bankruptcy, sell the company, massive layoffs	Revenue miss, CIO's job	Negligible

(Source: Adrian Cockcroft, "Velocity and Volume (or Speed Wins)," presentation at FlowCon, San Francisco, CA, November 2013.)



History of DevOps

- Several IT “movements” in the last 20 years
- Lean movement from manufacturing
- Agile Movement
- Agile Infrastructure movement
- CD – continuous delivery movement



Old Way - Stage of Software Development

- Build and test code
- Deploy and test software
- Release software or service



DevOps

- Integration and automation of....
 - Software development
 - IT operations processes
- Accelerates software delivery and improve overall efficiency



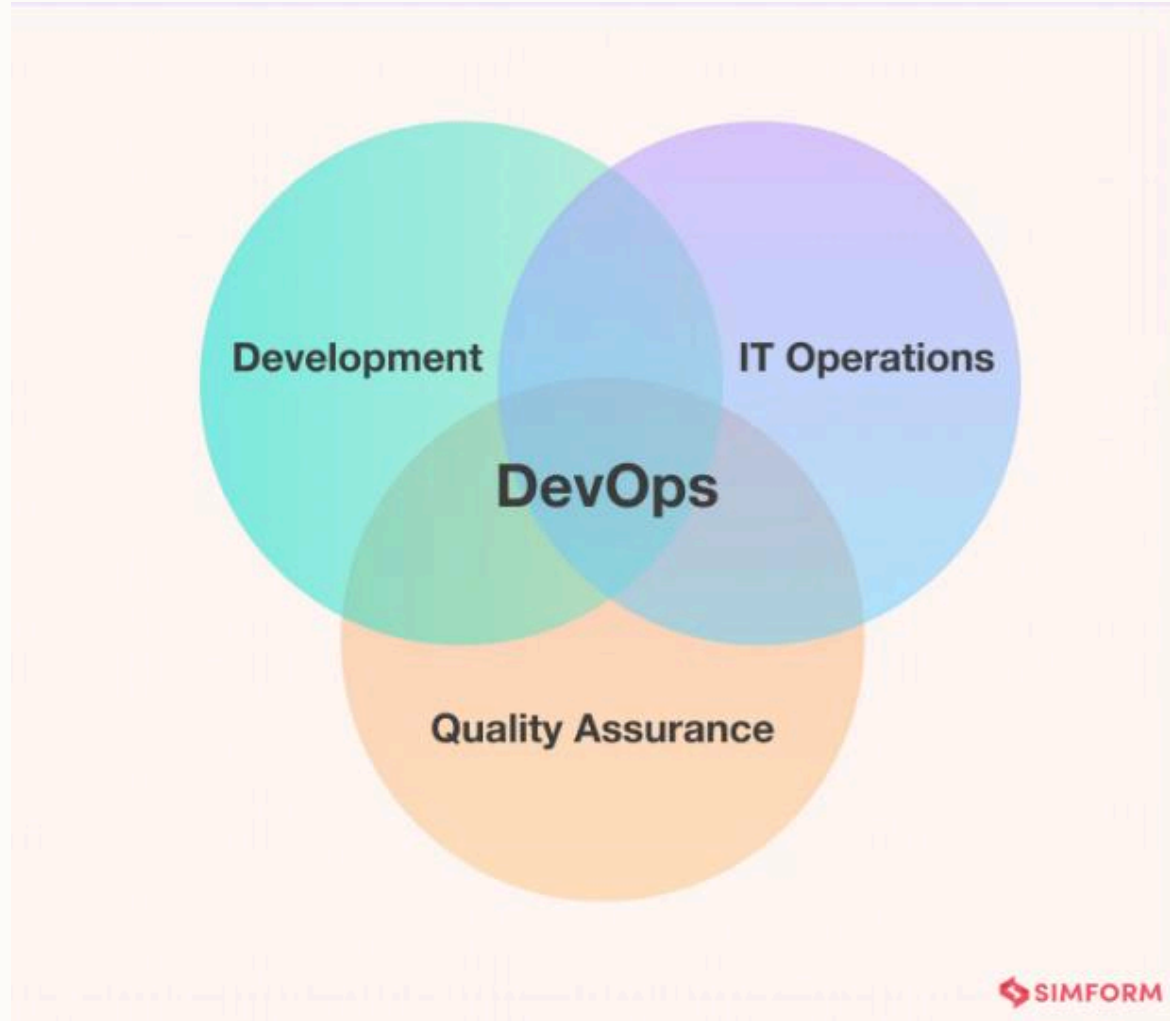
What is DevOps?

important

“The practice of operations and development engineers participating together through the entire service lifecycle, from the design and development process all the way to production support”



What is DevOps





What is DevOps – Core values

- Culture – Change people's behavior
- Automation – to accelerate change
- Measurement for improvement
 - Mean time to restore services
 - Cycle time to deploy a new features
 - Costs and revenues
 - Employer satisfaction
- Sharing
 - Teamwork, trust, and transparency, collaboration



Why DevOps

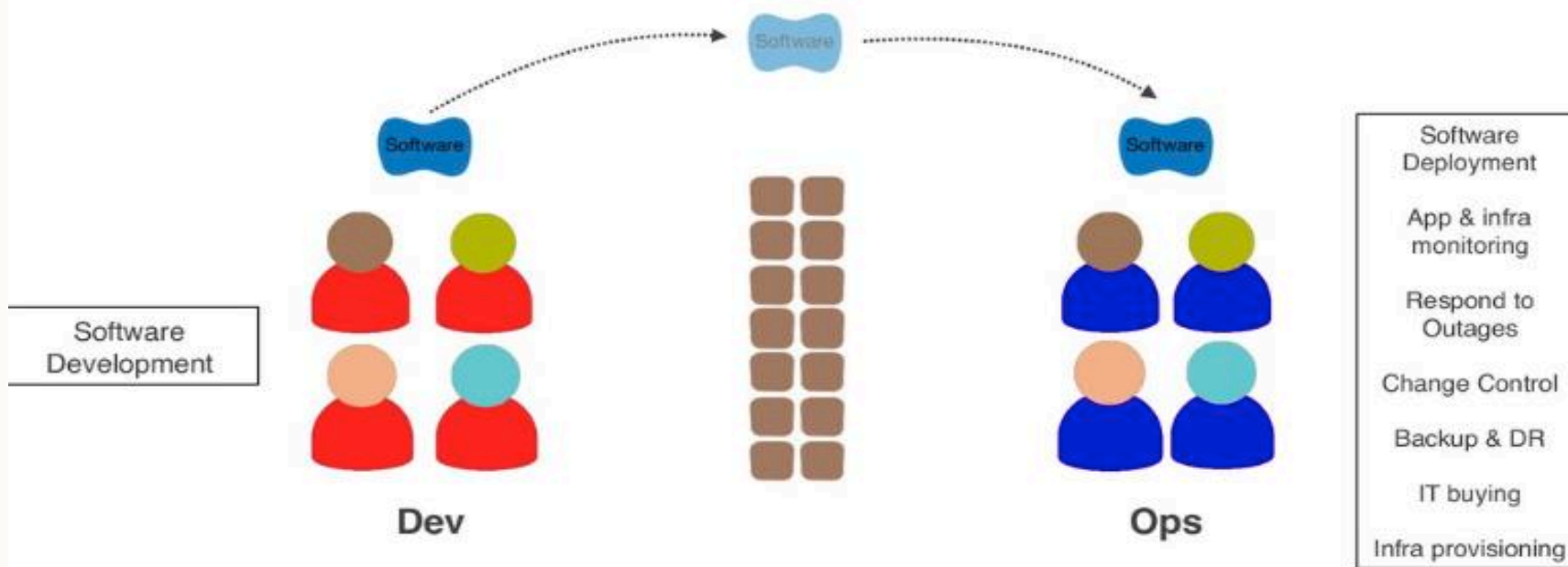
- Effective in improving IT and business outcomes
- Elite team deploy 973 times more frequently with 6,570 times shorter lead time than low IT performers.
- Heightened productivity cuts burnout in half

<https://dora.dev/research/2024/dora-report/2024-dora-accelerate-state-of-devops-report.pdf>



Culture Breaking down the Silos

#0 - Dev & Ops silo



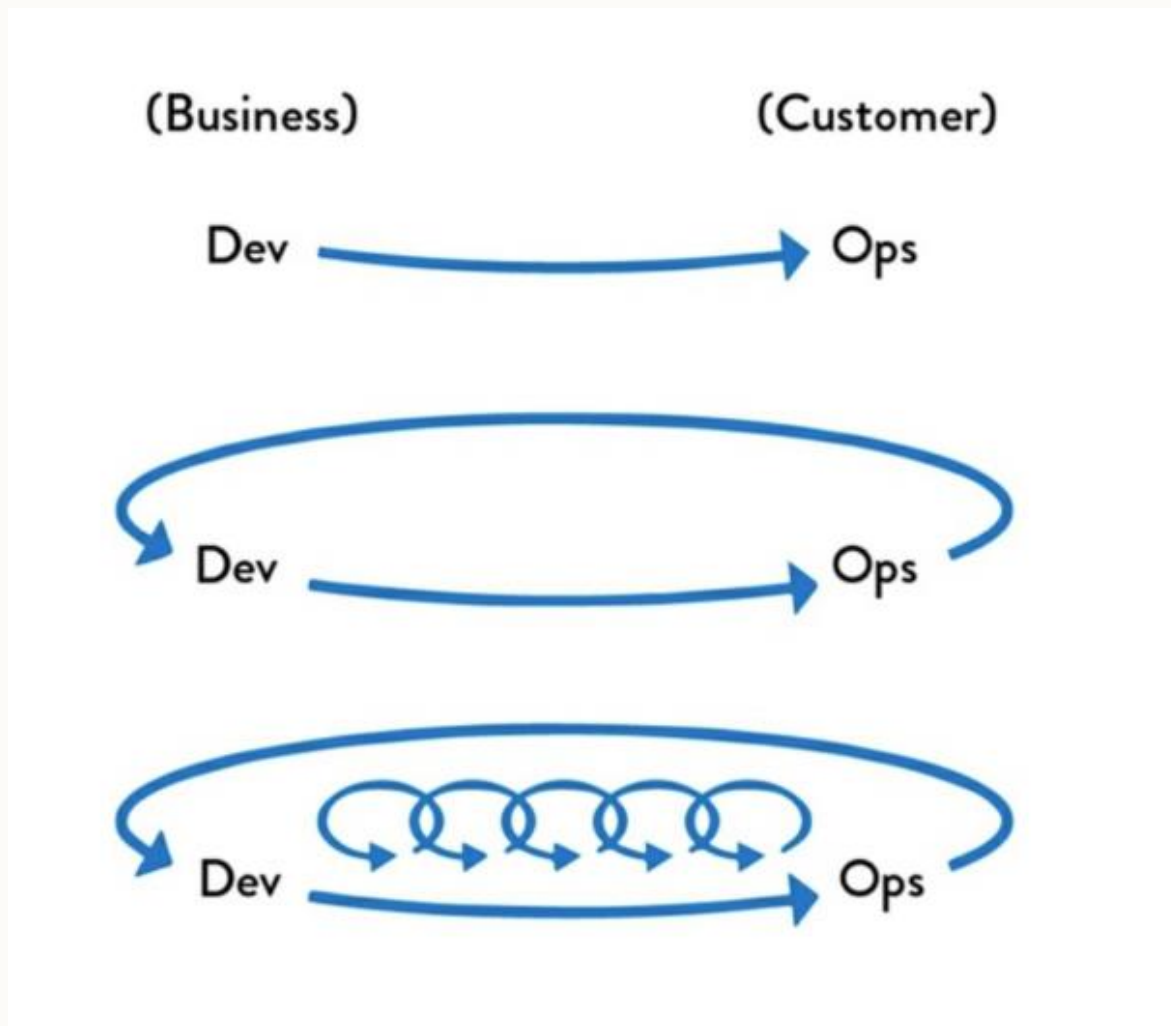


The Three Ways

1. Systems Thinking and the Principle of Flow

2. The Second Way – Amplify Feedback

3. A Culture of Continuous Experiment and Learning





The First Way - Systems Thinking and the Principle of Flow

- Making work “visible”
- Limiting work-in-progress (WIP)
- Reducing batch sizes
- Reducing hands-off between teams
- Continually identify and elevate our constraints

(Business)

(Customer)

Dev



Ops



The First Way : Making Work Visible

- Kanban board
- Unlike manufacturing world
 - Work of SE is invisible
- Left to right
- Work is not done when you finished a development ticket.
 - Why?
- Alignment with Agile?

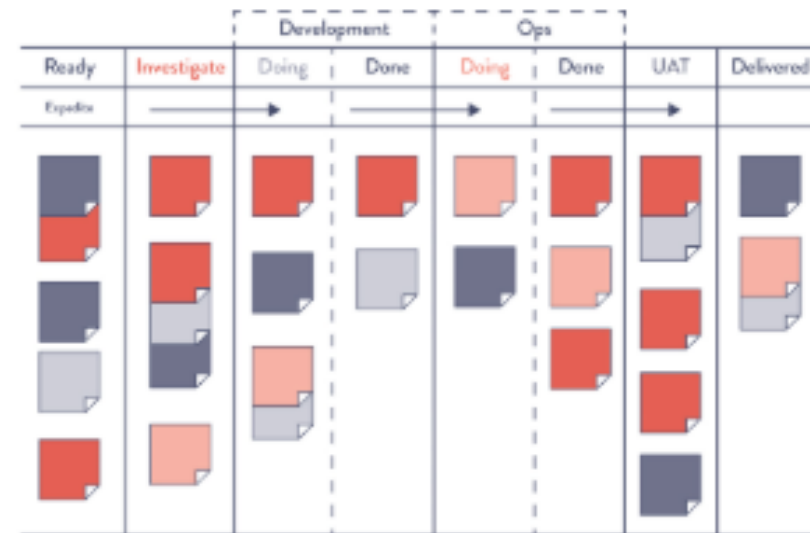


Figure 2.1: An Example Kanban Board Spanning Requirements, Dev, Test, Staging, and In Production

Source: David J. Andersen and Dominica DeGrandis, *Kanban for IT Ops*, training materials for workshop, 2012.



The First Way: Limit Work in Process(WIP)

- Challenges *priority change all the time*
 - Priority du jour from abundant communication channels
 - Multi-tasking is not good – prioritization problem
 - WIP is also reduced lead time
 - Inevitable loss of knowledge



The First Way: Reduce Batch Size

LARGE BATCHES



SINGLE PIECE FLOW



Software Engineering

- Batch of code
- Branch-based development
- Varies from the team to team
- From each commit to tested, deployed to production



Reduce the Number of Handsoff - Conway's law

"Organizations which design systems, are constrained to produce designs which are copies of communication structures of these organization"

Melvin Conway (1960)



Reduce the Number of Handsoff

- What are the operations required from version control to production environment?
 - Functional testing
 - Integration testing
 - Environment creation
 - Server administration
 - Storage administration
 - Networking
 - Load balancing
 - More and more



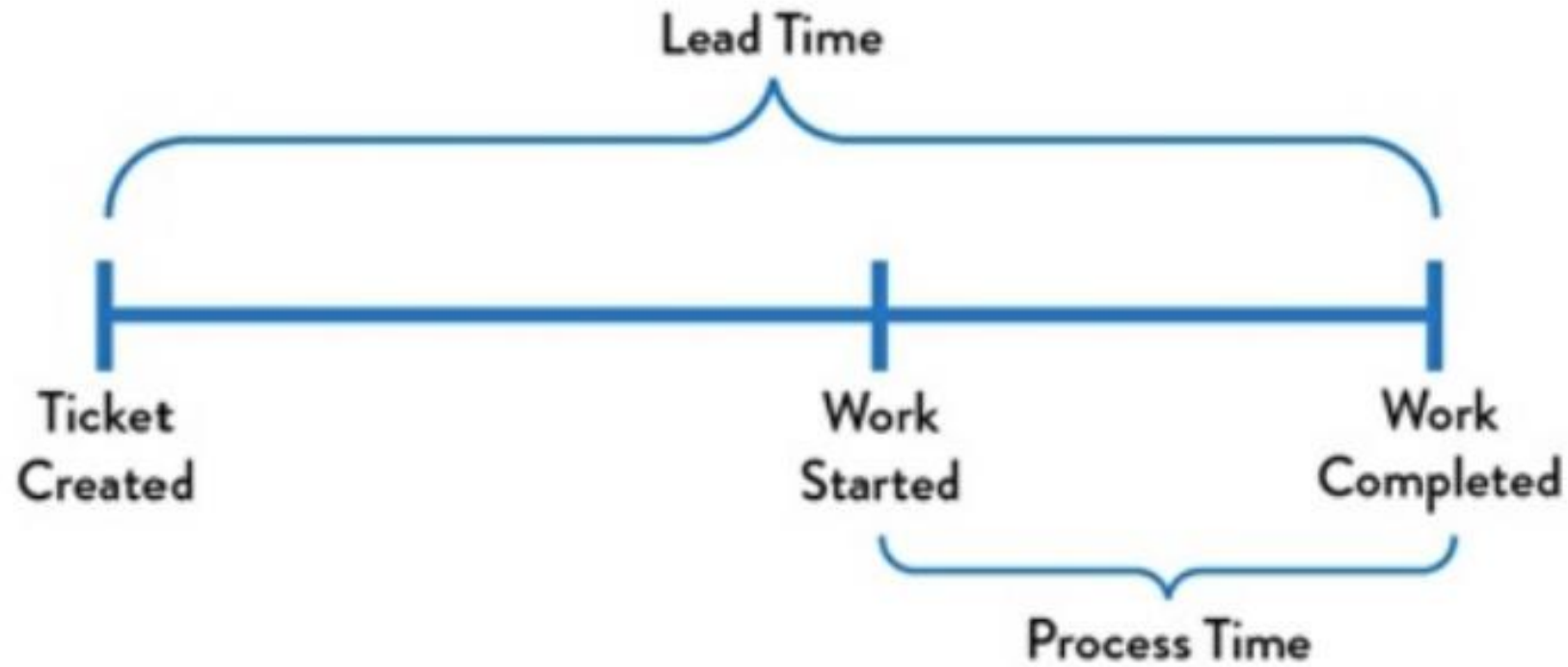
Reduce the Number of Handsoff

- What types of communication required?
 - Requesting
 - Specifying
 - Coordinating
 - Prioritizing
 - Scheduling
 - Deconflicting
 - More and more

Solution: Reduce number of handoff + reduce dependencies + automate = reduce number of non value added (Lean)



Process Time vs. Lead time





Deployment Lead Time

- What are the conditions for this to happen?
 - Architecture is modular
 - Well encapsulated
 - Loosely coupled
 - Small failure
 - Self-contained

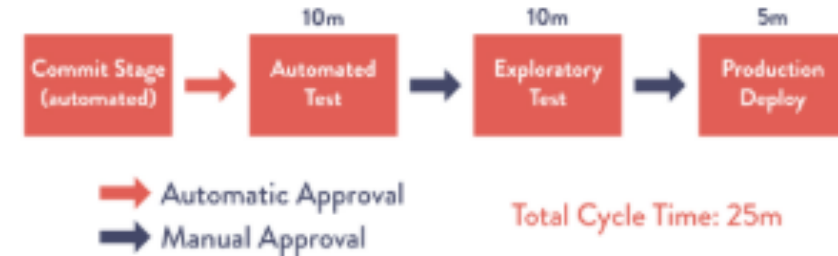
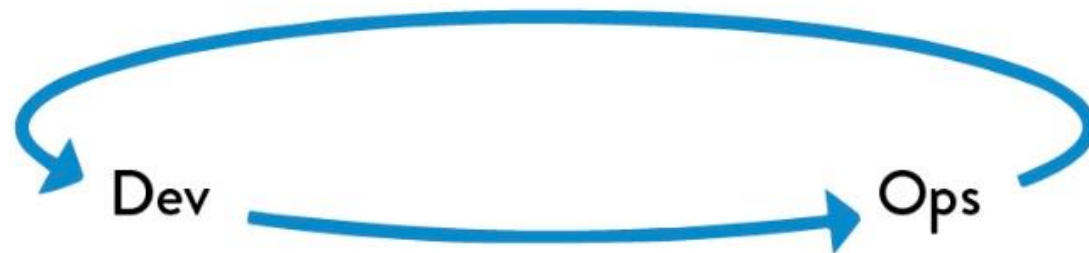


Figure 1.3: A Technology Value Stream with a Lead Time of Minutes



The Second Way – Amplify Feedback

- Enables fast and constant flow of feedback from **right to left** at all stages of our value stream
- Requires that we amplify feedback to prevent problems from happening again or enable faster detection of recovery
- We create quality at the source and generate or embed knowledge where it is needed
- We see problems as they occur and swarm theme until effective counter measures are in place





What are different ways to get feedback



Increase quality of work
through peer views and
pair programme



Code review



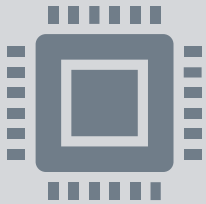
Learn from failures of
deployments



Integrate user research
and feedback into the
work of the product



Other examples



APPLICATION LEVEL: TRANSACTION
TIME, USER RESPONSE TIME



INFRASTRUCTURE LEVEL: WEB
SERVER TRAFFIC

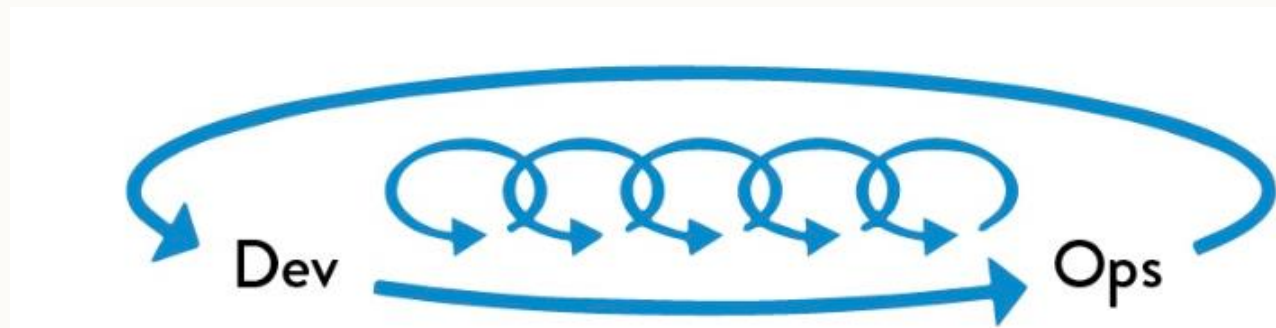


DEPLOYMENT PIPELINE LEVEL:
RED/GREEN AUTOMATED TEST
SUITES, DEPLOYMENT FREQUENCIES



The Third Way – Continuous Learning and Experimentation

- High-trust culture
- Creation of individual knowledge
- Organizational knowledge
- Leaders reinforce a learning culture
- Inject production failure to create resilience





Organizational Types

Pathological	Bureaucratic	Generative
Information is hidden	Information may be ignored	Information is actively sought
Messengers are "shot"	Messengers are tolerated	Messengers are trained
Responsibilities are shirked	Responsibilities are compartmented	Responsibilities are shared
Bridging between teams is discouraged	Bridging between teams is allowed but discouraged	Bridging between teams is rewarded
Failure is covered up	Organization is just and merciful	Failure causes inquiry
New ideas are crushed	New ideas create problems	New ideas are welcomed



Lesson from Google

- Data-driven decision: data from code, logs, traces to make decisions
- Transparency: sharing over secrecy and siloing – one ground truth
- Shared goals: one common goal for developer and operations
- No-blame policy : blameless postmortem aim is to build trust
- Education: learning organization; promote start up projects



DevOps Check-in

Instruction: Distribute 4 points across A-D.

Concerning the people aspect of DevOps: Adopting DevOps requires the leaders of the organization to work with their teams to create a DevOps Culture. Which of the following should be encouraged? (Multiple answers allowed)	My points	Team points
A. Collaboration across roles		
B. Reward developers for new features delivered		
C. Reward operations team for uptime and stability		
D. Visibility into project's goal and status		



Introduction to Continuous Integration and Continuous Delivery



Continuous Integration

Continuous Integration is a development practice that requires developers to integrate code into a shared repository several times a day. Each check-in is then verified by an automated build, allowing teams to detect problems early on.

Continuous Integration doesn't get rid of bugs, but it does make them dramatically easier to find and remove. – Martin Fowler



Why Continuous Integration

- Differences between Dev and production environment
- Cross-browser or cross-version testing
- Hard to protect against security threats
- Stress testing/longevity testing of new feature/code paths
- Example: Salesforce CI runs 150K + tests and automatically opens bug reports when test fails.



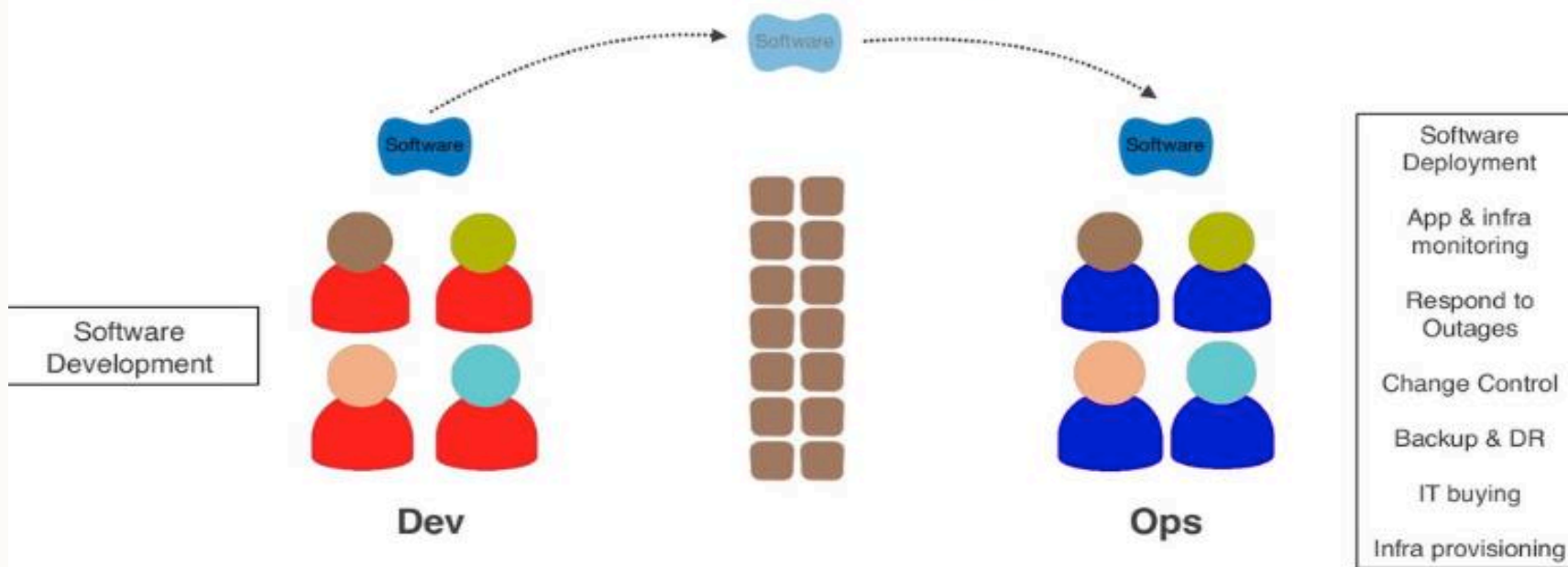
Key Terms to know about DevOps

- Software deployment – technical process of focused on coding running effectively in the environments but before the release
- Software release – to the end users



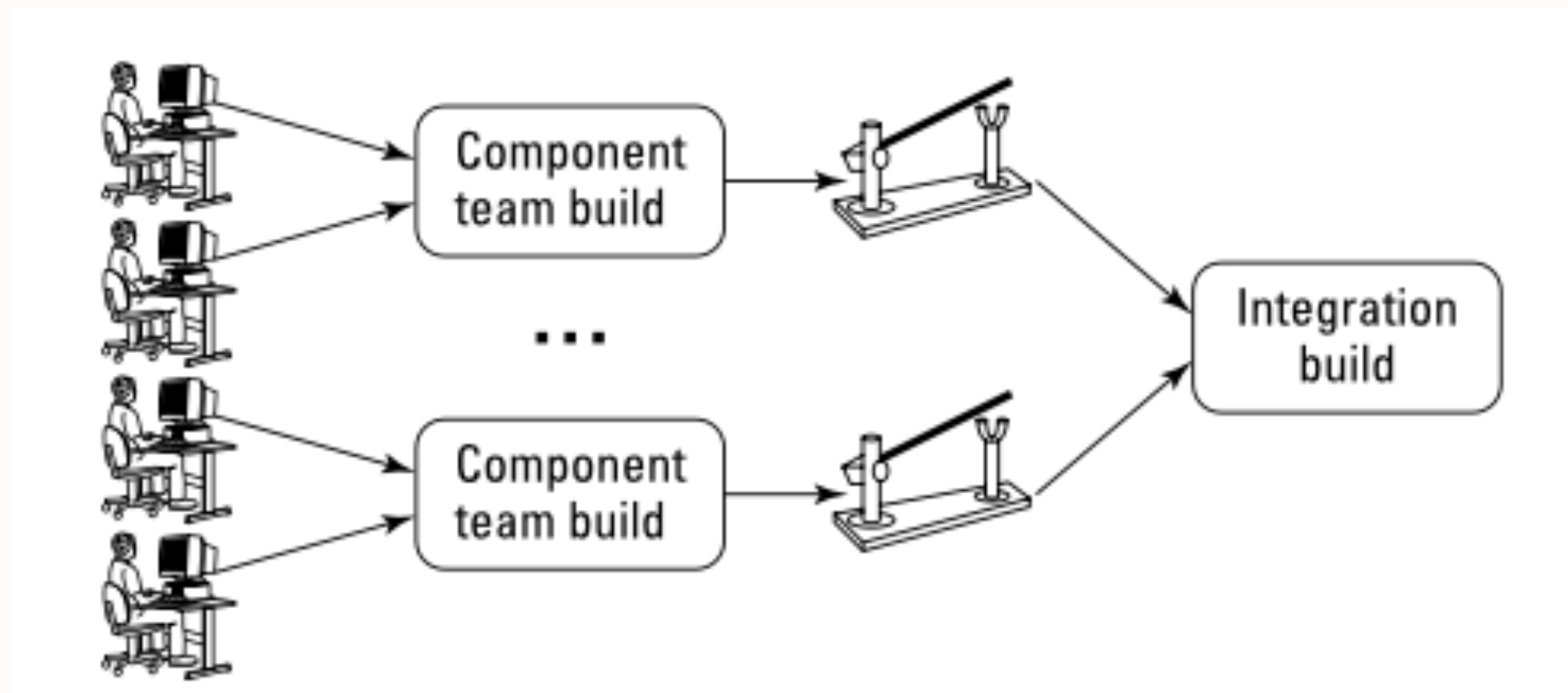
Culture Breaking down the Silos

#0 - Dev & Ops silo





Collaborative Development



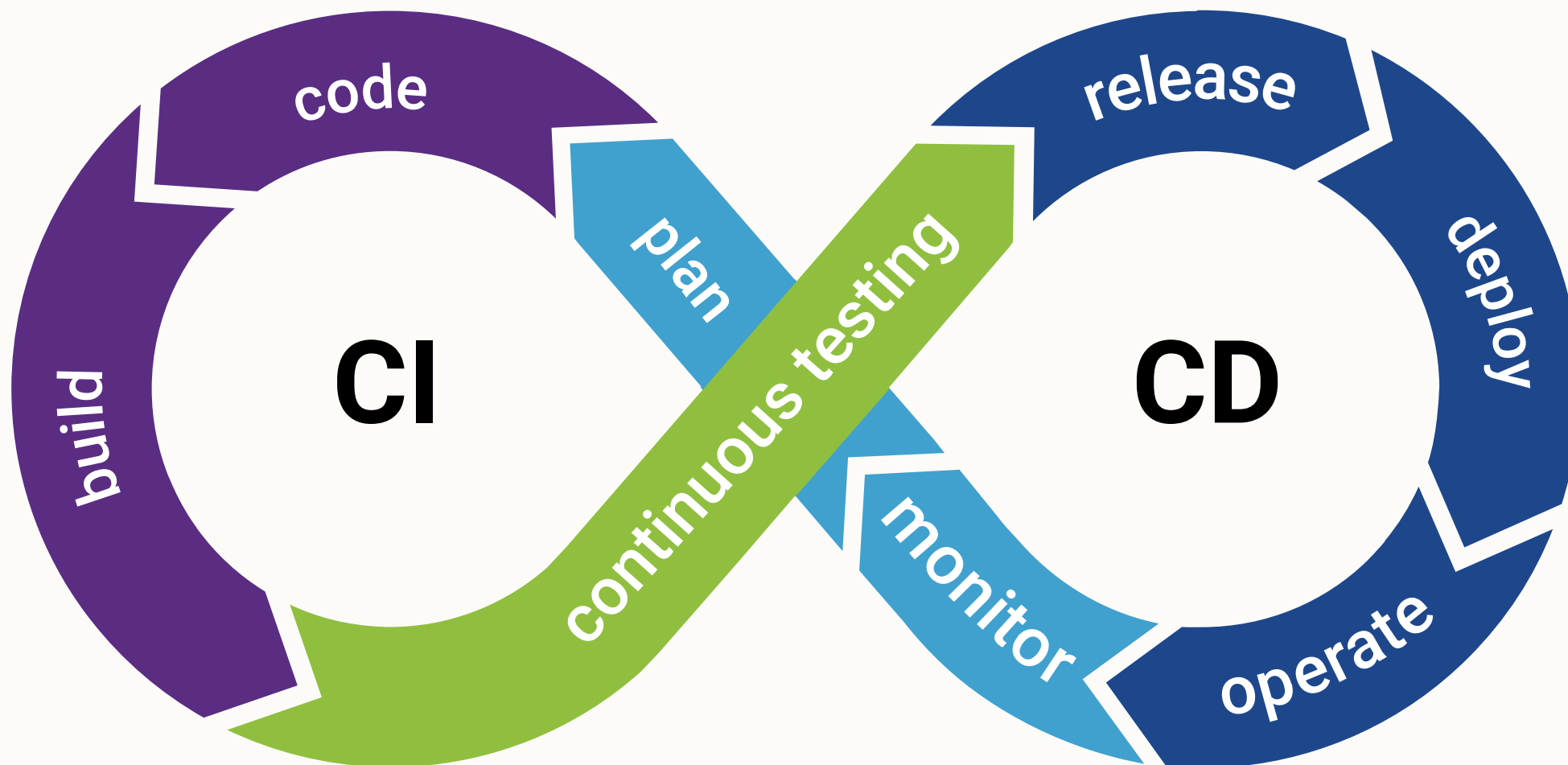


Continuous Delivery

The ability to get changes – features, configuration changes, bug fixes, experiments - into production or into the hands of users safely and quickly in a suitable way.



CI/CD



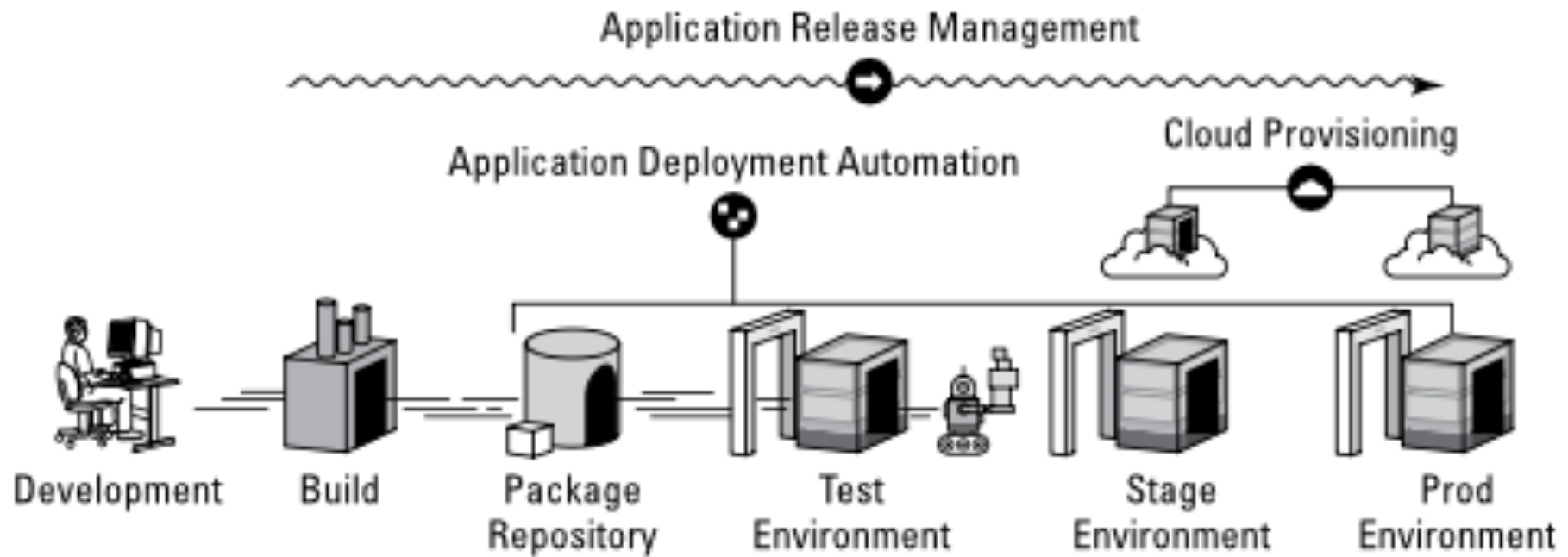


DevOps pipeline



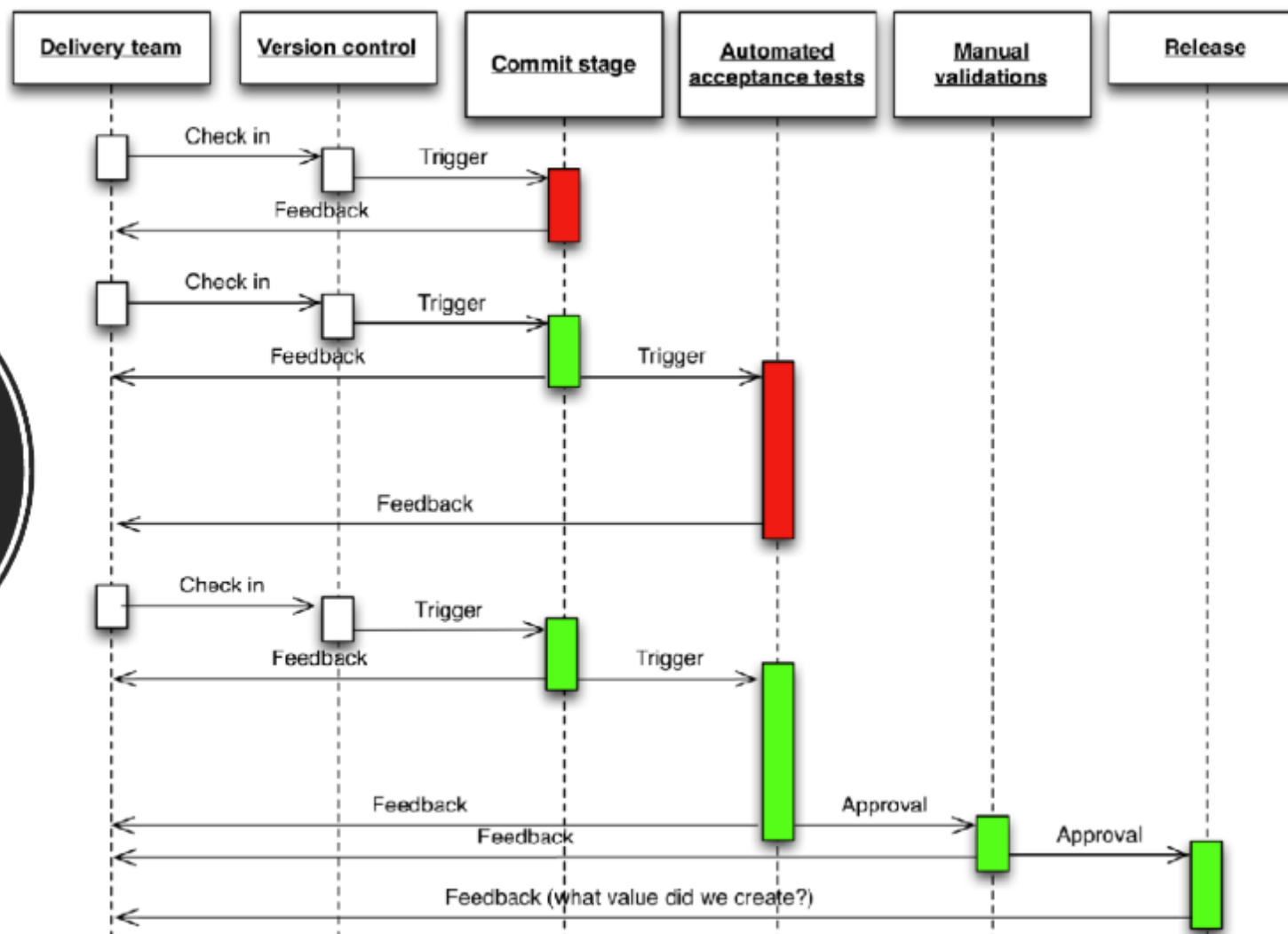


DevOps pipeline



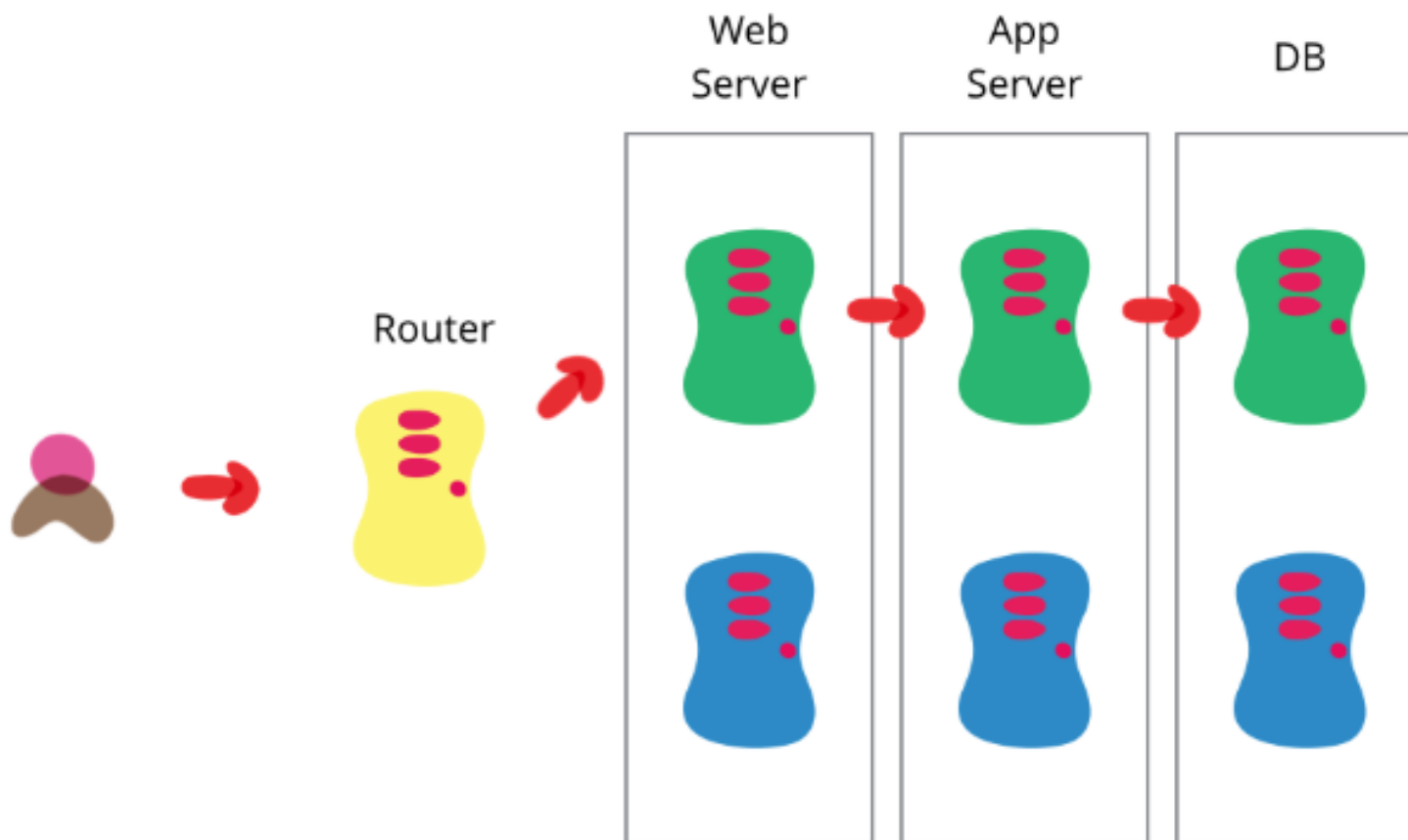


Deployment pipeline (Jez Humble)





Blue Green Deployment



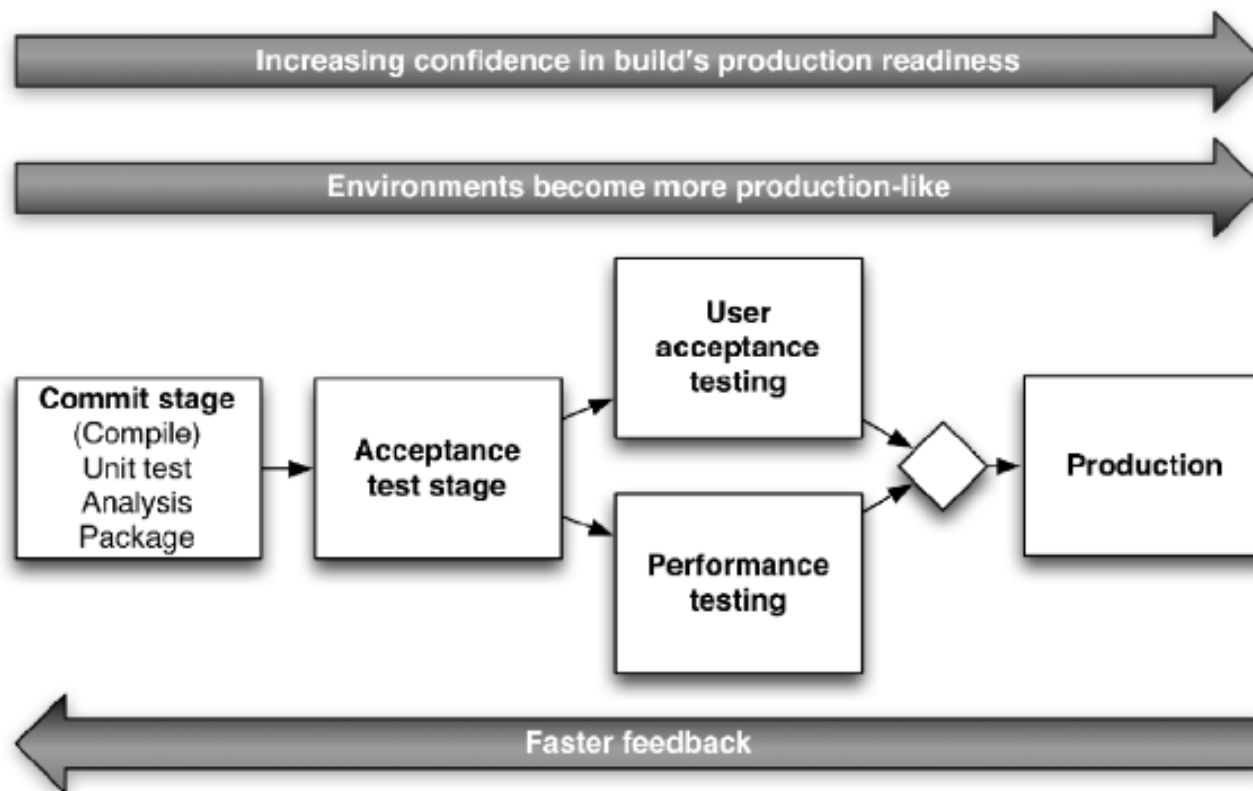


Benefits of CI/CD

- Automation of the software release process i.e. painless
- Lower testing costs because of multiple tests being run in seconds.
- Bug issues are quickly identified and resolved.
- The release time of the software is faster to reach the market through continuous testing and development.
- In cloud environments, Continuous Delivery allows teams to manage cloud resources effectively.
- Improved accessibility and communication in teams.
- Increased customer and employee satisfaction



What is the trade off?





Key high level measurements to improve CI/CD

- Deployment frequency
- Lead time for change
- Meant time to recover
- Change of failure rate



Key high level measurements to improve CI/CD

- **Deployment frequency** - Automated builds and testing are a need for automated deployment.
- **Lead time for change** - Build/test automation and early feedback assist in reducing the amount of time it takes to get code from commit to successfully operating in production.
- **Mean time to recover** – Average time it takes to restore from failure of the system.
- **Change of failure rate** - The number of faults that reach production is substantially decreased by early automated testing.



Key high level measurements to improve CI/CD



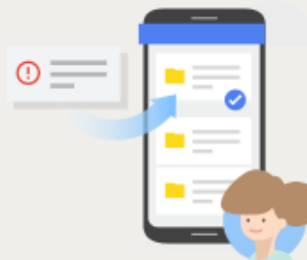
Change lead time:
the time it takes for a code commit or change to be successfully deployed to production.



Deployment frequency:
how often application changes are deployed to production.



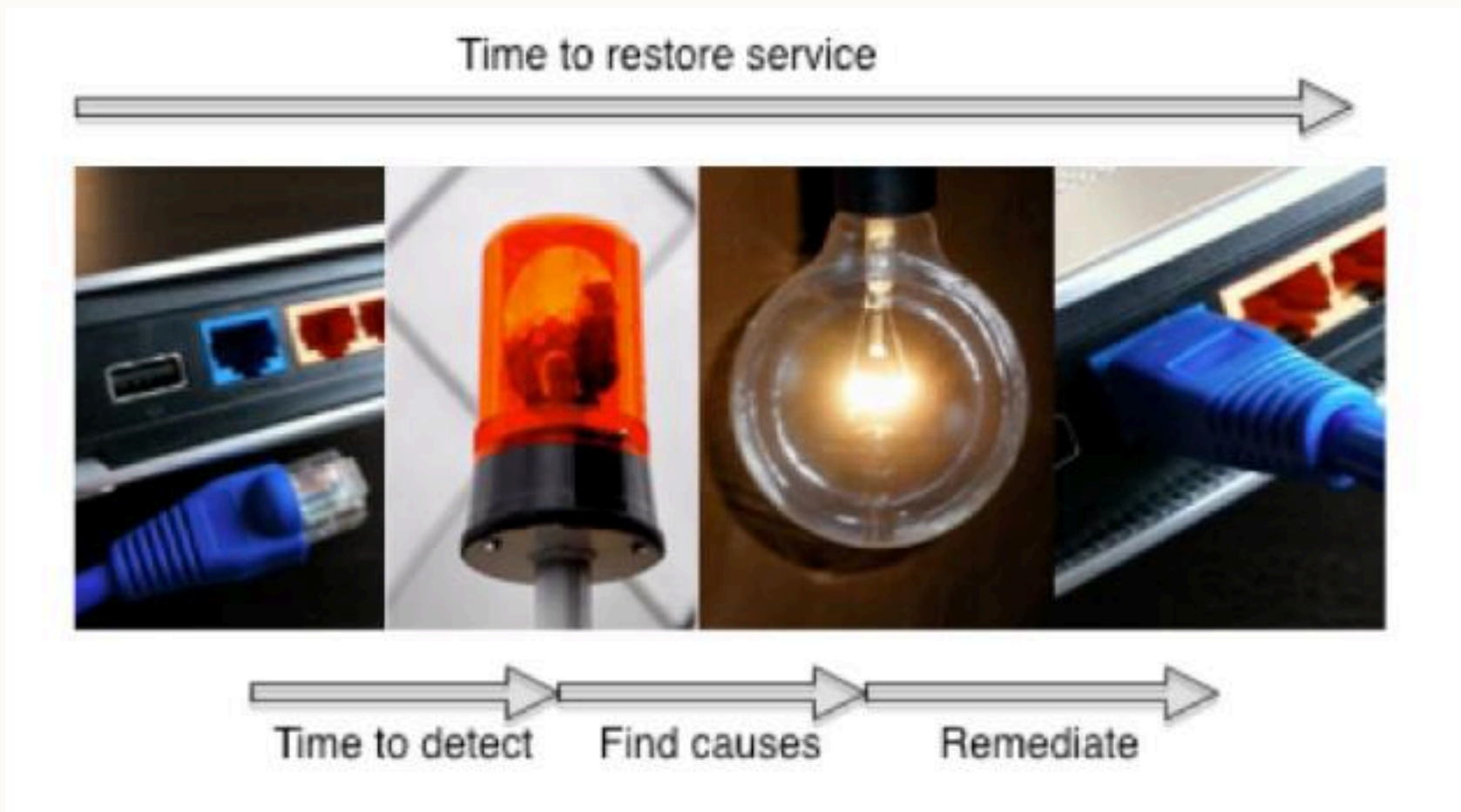
Change fail rate:
the percentage of deployments that cause failures in production,¹ requiring hotfixes or rollbacks.



Failed deployment recovery time:
the time it takes to recover from a failed deployment.



Optimize for Mean Time Recover (MTR)





DevOps Check-in

Instruction: Distribute 4 points across A-D.

It is important to identify bottlenecks in the delivery pipeline. Which of the following is not ones of the biggest sources of inefficiencies in the delivery pipeline?	My points	Team points
A. Unnecessary overhead (having to repeatedly communicate the same information and knowledge)		
B. Unnecessary rework (defects being uncovered in testing or production forcing assignments back to the development team)		
C. Unnecessary planning (too short sprints that makes the ratio between working effort and planning effort inefficient)		
D. Over-production (functionality developed that wasn't required)		



Performance level	Change lead time	Deployment frequency	Change fail rate	Failed deployment recovery time	Percentage of respondents*
Elite	Less than one day	On demand (multiple deploys per day)	5%	Less than one hour	19% (18-20%)
High	Between one day and one week	Between once per day and once per week	20%	Less than one day	22% (21-23%)
Medium	Between one week and one month	Between once per week and once per month	10%	Less than one day	35% (33-36%)
Low	Between one month and six months	Between once per month and once every six months	40%	Between one week and one month	25% (23-26%)



A few good quotes...

- **"The bottleneck of today is no longer just technical practices (though they still exist); the biggest challenge and necessity is getting business leadership on board"**
- **"Transformation must be co-created between the business and technology..."**
- **"We must achieve true collaboration. 90% of that work involves getting the right people engaged, onboard, and aligned"**



Timetable of Activities



Dev

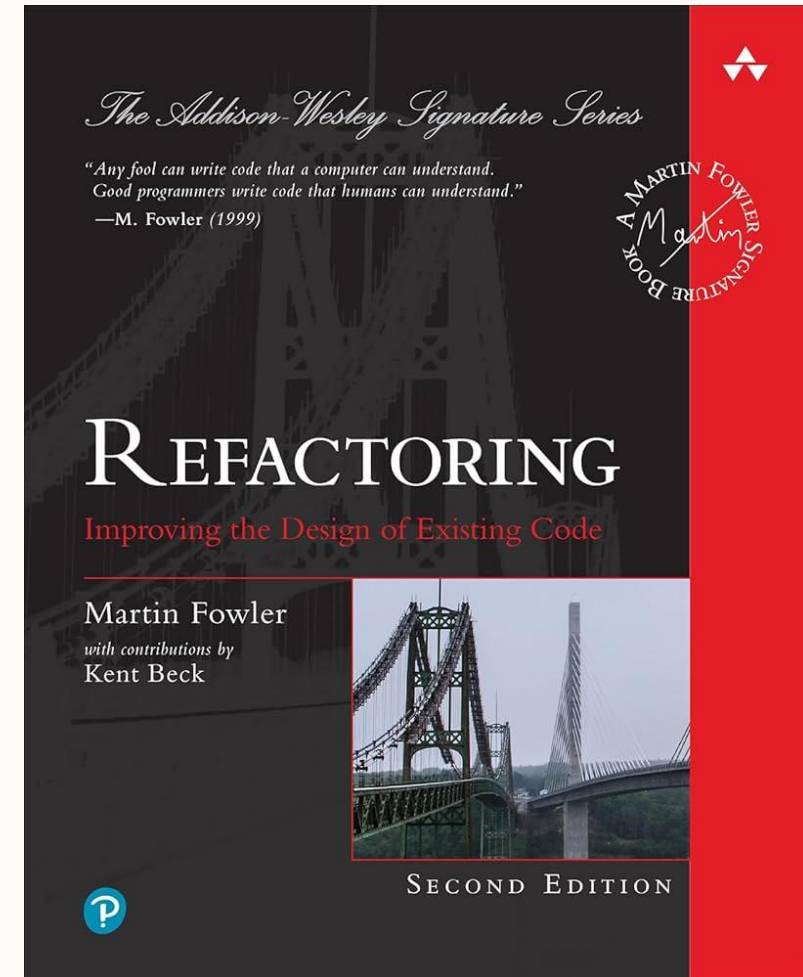
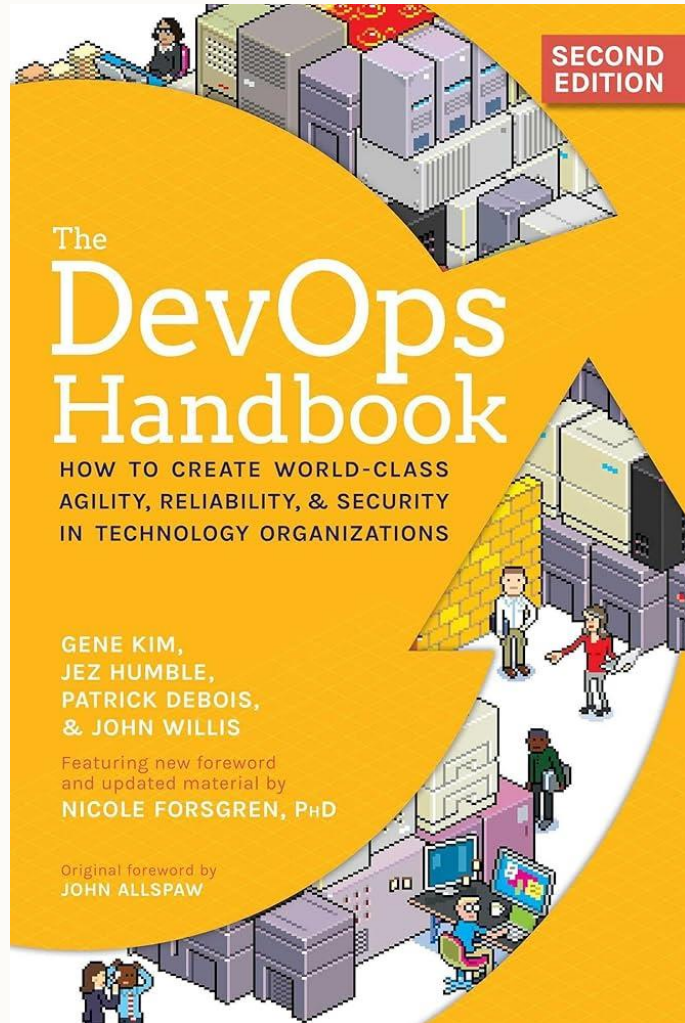
Ops

Week	Beginning	Lecture - Tue 2-4pm	Lecture includes...	Lab - Thu 3-6pm
1	Mon 28th Sept	Module Introduction	Industry Experience	Refreshing your OO Skills
2	Mon 5th Oct	Introduction to DevOps	Continuous Integration & Deployment	DevOps Scenario Comparisons
3	Mon 12th Oct	Refactoring 1: (Dev)	Identifying Code Smell in Legacy Code; Swing; CW Release	Refactor 1
4	Mon 19th Oct	Refactoring 2: (Dev)	Techniques to remove code smells;CapitalOne - AI in Industry	Refactor 2
5	Mon 26th Oct	Refactoring 3: (Dev)	Coding principles ,Maintenance Testing ,JavaFx testing Maven intro	Refactor 3
6	Mon 2nd Nov	Refactoring 4 : (Dev)	Design Patterns ; Swing and Java Fx GUIs	Refactor 4
7	Mon 9th Nov	DevOps Infrastructure as Code 1(Ops)	TestFX demo; Java Doc demo ; IntelliJ demo	Refactor 4
8	Mon 16th Nov	DevOps Infrastructure as Code 2(Ops)	Industry Pipeline Demo	CI/CD Pipeline Exercise
9	Mon 23rd Nov	Software Architecture (DevOps)	Industry Panel(tbc)	Code Review Exercise
10	Mon 30th Nov	Advanced DevOps	DevSecOps ; Docker & Containers	Coursework Support
11	Mon 7th Dec	AI in Software Maintenance(DevOps)	Final CW Q&A	Coursework Support

NOTE: some aspects of the timetable are subject to change.



Textbooks





Reading this week

- Read Chapter's 1, 2 and 3 in The DevOps Handbook – How to Create World-Class Agility, Reliability and Security in Technology Organisations by Gene Kim, Jez Humble, Patric Debois and John Willis.
- This is available in the library:
<https://rl.talis.com/3/notts/lists/EDCA0F02-AE7C-304A-7596-DDEB8806EF3A.html?lang=en-GB>



Lab 02

- Read Chapter 1, 2 and 3 from The DevOps Handbook
- Practice strategies for implementing principles of DevOps.



Learning Outcomes for this Week

- At the end of this week ...
 - Understand what is DevOps in terms of principles
 - Be able to apply role of business value and people in the organisation to create values through Dev and Ops
 - Understand how to operationalize DevOps in a software companies through CI/CD
 - Practice some of the DevOps principles through different scenarios





References and Acknowledgements

- The DevOps Handbook – How to Create World-Class Agility, Reliability and Security in Technology Organisations by Gene Kim, Jez Humble, Patrick Debois and John Willis
- Martinflowler.com
- <https://learn.gitlab.com/>
- Michael Verdicchio, Department of Cyber and Computer Sciences The Citadel Charleston, SC – from his course on DevOps
- Check this online course out: <https://www.coursera.org/learn/devops-culture-and-mindset?action=enroll>
- DevOps Summer school – University of Southern Denmark by Peter Lorenz Nellemann
- <https://speakerdeck.com/shahadarsh/role-of-team-structures-in-devops-transformation-journey-ignite-version?slide=4>
- <https://www.civo.com/blog/the-role-of-the-ci-cd-pipeline-in-cloud-computing>

Lecture 02 - DevOps & CI/CD Core Revision Summary

DevOps principles · the Three Ways · flow / WIP / batch size / handoffs · feedback and learning culture · CI/CD · delivery metrics

◆ lecture/discussion-emphasized content

1. Topic Overview

This lecture introduces DevOps as the integration of software development and IT operations across the service lifecycle. The main thread is how organisations create business value by improving flow, shortening feedback loops, encouraging continuous learning, and operationalising these ideas through CI/CD.

Lecture logic: Agile / Lean roots -> DevOps culture -> the Three Ways -> flow and lead time -> feedback and learning -> Continuous Integration / Delivery -> delivery performance metrics.

2. Core Concepts

What DevOps means

- ▶ DevOps integrates and automates software development and IT operations processes to accelerate delivery and improve overall efficiency.
- ▶ ◆ Lecture definition: development and operations engineers participate together through the entire service lifecycle, from design and development to production support.
- ▶ DevOps is therefore a way of organising people, process and technology; it is not just a deployment tool or job title.

Core values: Culture, Automation, Measurement, Sharing

- ▶ Culture: change behaviour; break down Dev/Ops silos; create trust, teamwork and collaboration.
- ▶ Automation: accelerate change and remove repetitive manual work.
- ▶ Measurement: use measures such as restore time, deployment cycle time, costs/revenues and satisfaction to drive improvement.
- ▶ ◆ Sharing: transparency, shared knowledge, common goals and collaboration across roles.

The Three Ways

1. Systems Thinking and the Principle of Flow - improve left-to-right flow from business / Dev toward Ops / customer.
2. Amplify Feedback - create fast right-to-left feedback at every stage of the value stream.
3. Continuous Learning and Experimentation - build a high-trust learning culture and improve resilience through experimentation.

Why DevOps matters

- ▶ The lecture frames software delivery as a trend toward faster, cheaper and lower-risk delivery.
- ▶ The objective is to improve both IT outcomes and business outcomes, rather than optimising Dev and Ops independently.

3. The First Way - Systems Thinking and Flow

The First Way focuses on the end-to-end value stream. Local team utilisation is less important than getting useful work safely to the customer with minimal waiting and rework.

1) Make work visible

- ▶ Software-engineering work is largely invisible, so use a Kanban-style board to make work and queues visible.

- ▶ ♦ Visualise work left to right across the full delivery path. A development ticket is not truly done if it is still waiting for testing, deployment or release.

2) Limit Work in Progress (WIP)

- ▶ Too much simultaneous work creates prioritisation problems, context switching and knowledge loss.

- ▶ ♦ Reducing WIP reduces lead time: finish existing work before starting more work.

Discussion shorthand: Too much WIP -> more multitasking -> more context switching / waiting -> more knowledge loss -> longer lead time.
Remember: "Stop starting; start finishing."

3) Reduce batch size

- ▶ Prefer small batches of code over large batches that wait for a long time before integration and release.

- ▶ In software this can mean moving from large branch-based batches toward a flow where each commit can be tested and deployed quickly.

- ▶ Smaller batches expose problems earlier and make failures easier to isolate.

4) Reduce handoffs

- ▶ Moving from version control to production can involve testing, environment creation, server/storage/network administration, load balancing and many other activities.

- ▶ Every handoff adds communication work: requesting, specifying, coordinating, prioritising, scheduling and deconflicting.

- ▶ ♦ Lecture solution: reduce handoffs + reduce dependencies + automate -> reduce non-value-added work (Lean).

- ▶ Conway's Law: organisations tend to produce system designs that mirror their communication structures.

5) Lead time vs. process time

Lead time: Ticket created -> Work completed
Process time: Work started -> Work completed

Lead time includes the waiting time before active work begins; therefore queues and WIP can dominate delivery time.

6) Deployment lead time - enabling architecture

- ▶ Short deployment lead time is supported by architecture that is modular, well encapsulated, loosely coupled and self-contained.

- ▶ ♦ Failures should have a small scope / small blast radius, allowing changes to be tested and deployed safely.

Architecture logic: independent components -> smaller changes -> easier testing -> smaller failures -> safer / more frequent deployment -> shorter deployment lead time.

4. The Second and Third Ways

The Second Way - Amplify Feedback

- ▶ Create a fast and constant flow of feedback from right to left at all stages of the value stream.
- ▶ Amplify feedback so problems are prevented from recurring or are detected and recovered from faster.
- ▶ Create quality at the source and place knowledge where it is needed.
- ▶ See problems as they occur and swarm them until effective countermeasures are in place.

Ways to obtain feedback

- ▶ Peer review and pair programming.
- ▶ Code review.
- ▶ Learn from failed deployments.
- ▶ Integrate user research and user feedback into product work.
- ▶ Measure at multiple layers: application transaction / response time, infrastructure traffic, and deployment-pipeline test / deployment signals.

The Third Way - Continuous Learning and Experimentation

- ▶ Build a high-trust culture.
- ▶ Create both individual knowledge and organisational knowledge.
- ▶ Leaders reinforce a learning culture.
- ▶ The lecture explicitly includes injecting production failure to create resilience.

Culture and organisational behaviour

- ▶ A generative culture actively seeks information, shares responsibility, rewards bridging between teams, investigates failures and welcomes new ideas.
- ▶ Data-driven decisions: use code, logs and traces.
- ▶ Transparency: sharing over secrecy / siloing; one ground truth.
- ▶ Shared goals: Dev and Ops should optimise for a common outcome.
- ▶ No-blame policy: blameless postmortems are intended to build trust.
- ▶ Education: support learning and experimentation.

DevOps culture check-in from the lecture:

Encourage A. collaboration across roles and D. visibility into the project's goal and status.

Rewarding Dev only for new features and Ops only for uptime creates conflicting local incentives and reinforces silos.

Operational implication

- ▶ A DevOps transformation requires business and technology leadership to be aligned; the lecture emphasises that the challenge is strongly about people engagement and collaboration, not only technical practices.

5. Continuous Integration and Continuous Delivery

Continuous Integration (CI)

- ▶ CI is a development practice in which developers integrate code into a shared repository several times a day.
- ▶ Each check-in is verified by an automated build, allowing problems to be detected early.
- ▶ CI does not eliminate bugs; it makes them easier to find and remove.
- ▶ Motivations discussed include Dev/production differences, cross-browser or cross-version testing, security threats, and stress/longevity testing.

Deployment vs. release

Software deployment: the technical process of getting code to run effectively in an environment, before release.

Software release: making the software available to end users.

Continuous Delivery (CD)

- ▶ CD is the ability to get changes - features, configuration changes, bug fixes and experiments - into production or into users' hands safely and quickly.
- ▶ A typical DevOps pipeline shown in the lecture is: Source Control -> Build -> Test -> Deploy -> Monitor & Feedback.
- ▶ The more detailed pipeline also includes package repositories, test / stage / production environments, deployment automation, cloud provisioning and application release management.
- ▶ Blue-green deployment is presented as one deployment technique.

Benefits of CI/CD

- ▶ Automates the software release process and makes releases less painful.
- ▶ Reduces testing cost by running many tests quickly.
- ▶ Finds and resolves bug issues earlier.
- ▶ Reduces time to market through continuous testing and development.
- ▶ Helps teams manage cloud resources effectively.
- ▶ Improves team communication / accessibility and can increase customer and employee satisfaction.

CI/CD trade-off

- ▶ Early pipeline stages provide faster feedback.
- ▶ As changes move toward production, environments become more production-like and confidence in production readiness increases.
- ▶ The practical trade-off is therefore feedback speed vs. production realism / confidence.

High-level delivery measurements

Metric	Lecture definition / interpretation
Deployment frequency	How often application changes are deployed to production; automated build/testing enables automated deployment.
Lead time for change	Time from code commit / change to successfully operating in production; build/test automation and early feedback reduce it.
Mean time to recover	Average time to restore service after a system failure.
Change failure rate	Percentage / number of deployments whose faults reach production and require remediation; early automated testing should reduce it.

6. Comparison & Common Confusions

Concept A	Concept B	Key Difference / Revision Point
Dev / Ops silos	Shared DevOps culture	Separate local incentives create conflict; DevOps favours collaboration, visibility, trust and shared goals.
Lead time	Process time	Lead time: ticket created -> completed. Process time: work started -> completed. Waiting time is part of lead time.
Large batch	Small batch	Large batches wait longer and hide problems; small batches reach testing / deployment sooner and reduce failure scope.
CI	Continuous Delivery	CI integrates and automatically verifies code frequently; CD focuses on safely and quickly getting validated changes toward production / users.
Deployment	Release	Deployment is the technical move into an environment; release makes the change available to end users.
Deployment frequency	Lead time / recovery / failure rate	Frequency measures how often; lead time measures delivery speed; recovery measures restoration speed; failure rate measures production-change quality.

7. Exam / Revision Focus

Must Know

- Definitions: DevOps, WIP, lead time, process time, CI, Continuous Delivery, deployment, release.
- The Three Ways and the direction of flow / feedback.
- First Way practices: make work visible, limit WIP, reduce batch size, reduce handoffs, identify / elevate constraints.
- CI pipeline idea: Source Control -> Build -> Test -> Deploy -> Monitor & Feedback.
- The four delivery metrics: deployment frequency, lead time for change, mean time to recover, change failure rate.

Must Understand

- Why reducing WIP and batch size can reduce waiting, lead time and the size of failures.
- Why loose coupling, modularity and encapsulation support short deployment lead time.
- Why reducing handoffs and dependencies removes non-value-added coordination work.
- How feedback loops, peer/code review, deployment learning and user feedback improve quality.
- Why DevOps culture uses shared goals and transparency instead of separate Dev-only and Ops-only incentives.

Easy to Confuse / Likely Quiz Traps

- Short lead time is not the same as short coding time: queues and waiting can dominate lead time.
- More WIP is not more throughput: multitasking and context switching can make completion slower.
- CI is not deployment: CI verifies frequent integration; later pipeline stages handle deployment / delivery.
- Deployment is not release: a system can be deployed before users receive the change.
- DevOps is not only automation: culture, measurement and sharing are equally explicit lecture values.
- Failure is not automatically “bad” in the Third Way: controlled experimentation and learning are used to build resilience.

Discussion emphasis incorporated: First Way / WIP logic, lead time vs. process time, deployment lead-time conditions, and the DevOps culture check-in.